

Data Warehouse Design (Logical design)

Letizia Tanca
Politecnico di Milano

with kind thanks to
Rosalba Rossato

Data Mart *logical models*

- **MOLAP** (*Multidimensional On-Line Analytical Processing*) stores data by using a multidimensional data structure
- **ROLAP** (*Relational On-Line Analytical Processing*) uses the relational data model to represent multidimensional data

Data Mart logical models: MOLAP

MOLAP (Multidimensional OLAP) is the more traditional way of OLAP analysis. Data is natively stored in a multidimensional cube. The storage is not in the relational database, but in proprietary formats.

ADVANTAGES: Excellent performance:

MOLAP cubes are built for fast data retrieval, and are optimal for slicing and dicing operations. Can perform complex calculations: All calculations have been pre-generated when the cube is created. Hence, complex calculations are not only doable, but they return quickly.

DISADVANTAGES : Limited in the amount of data it can handle

Because all calculations are performed when the cube is built, it is not possible to include a large amount of data in the cube, therefore only summary-level information will be included in the cube itself. Usually more than 90% of cells are empty → issue with sparsity. Requires additional investment, since cube technology are often proprietary and do not already exist in the organization. Therefore, to adopt MOLAP technology, additional investments in human and capital resources are needed.

Data Mart logical models: Rolap (the de-facto standard)

ROLAP (Relational OLAP): relies on manipulating the data stored in the relational database to give the appearance of traditional OLAP's slicing and dicing functionality. In essence, each action of slicing and dicing is equivalent to adding a "WHERE" clause in the SQL statement.

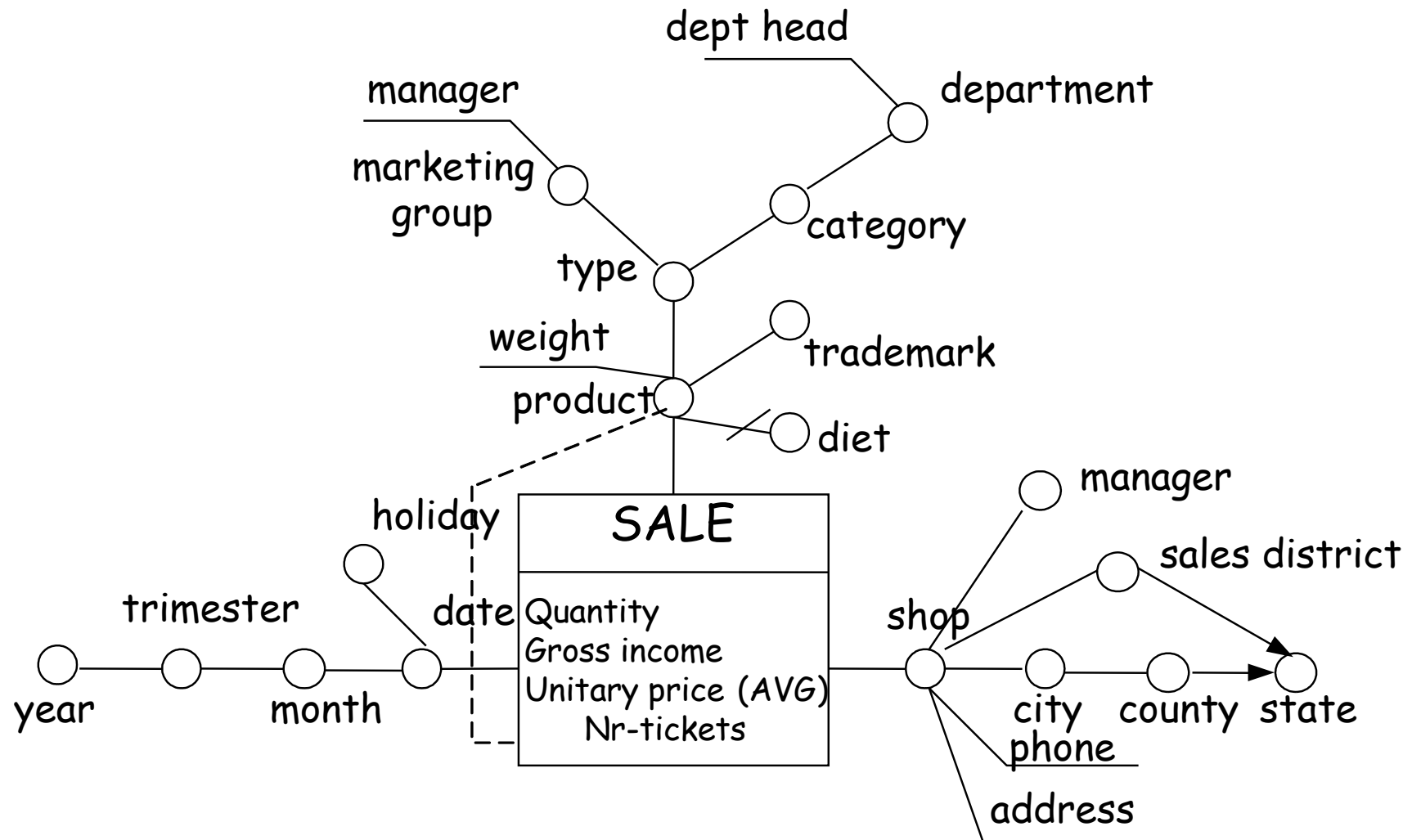
Advantages: Can handle large amounts of data: The data size limitation of ROLAP technology is the limitation on data size of the underlying relational database. Can leverage functionalities inherent in the relational databases.

Disadvantages: Performance can be slow: Because each ROLAP report is essentially a SQL query (or multiple SQL queries) in the relational database, the query time can be long if the underlying data size is large. Limited by SQL functionalities: Because ROLAP technology mainly relies on generating SQL statements to query the relational database, and SQL statements do not fit all needs (for example, it is difficult to perform complex calculations using SQL), ROLAP technologies are therefore traditionally limited by what SQL can do. ROLAP vendors have mitigated this risk by building into the tool out-of-the-box complex functions as well as the ability to allow users to define their own functions.

HOLAP (Hybrid Holap): attempts to combine the advantages of MOLAP and ROLAP. For summary-type information, HOLAP leverages cube technology for faster performance. When detail information is needed, HOLAP can "drill through" from the cube into the underlying relational data.

Data Warehouse Logical Design in ROLAP

Our fact schema example



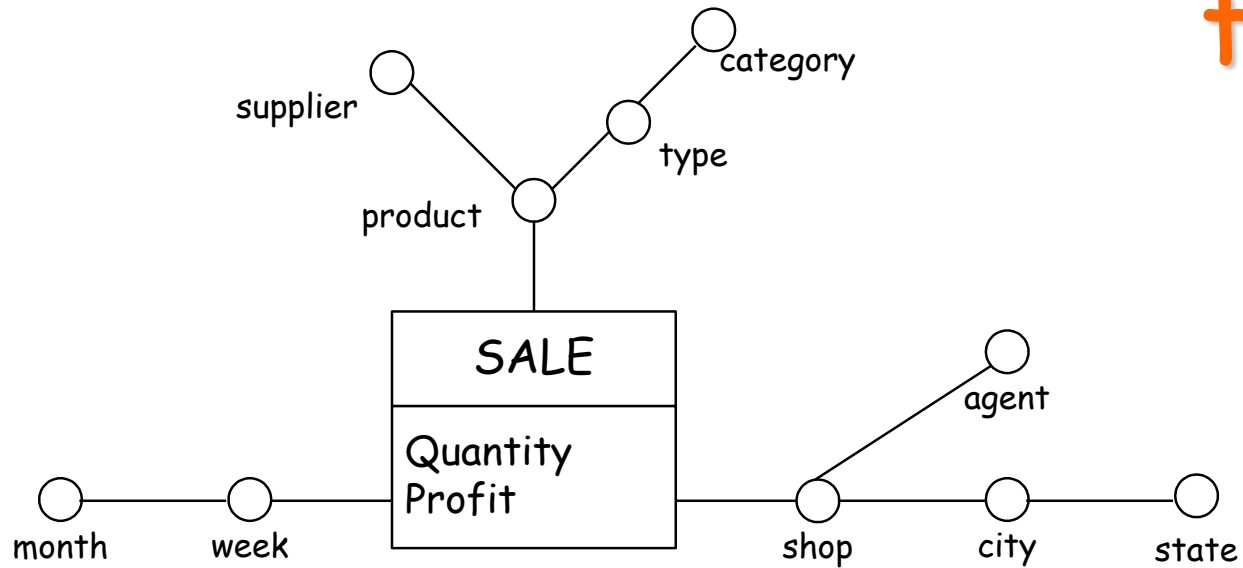
A de-facto standard for the logical model is ROLAP

It is based on the **Star Schema**

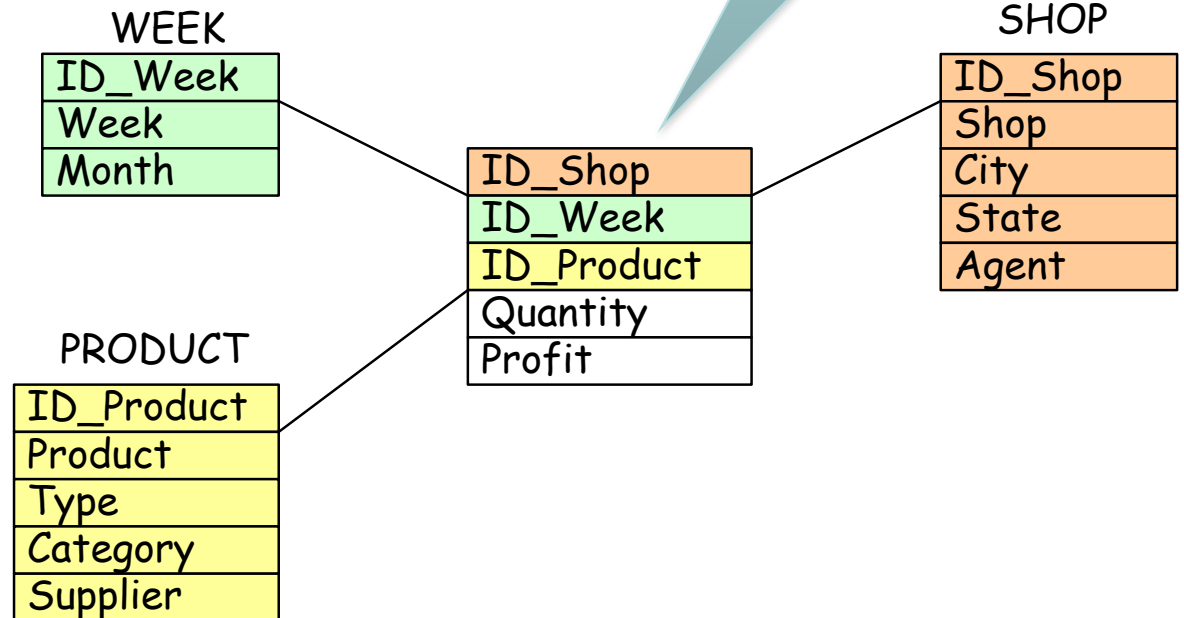
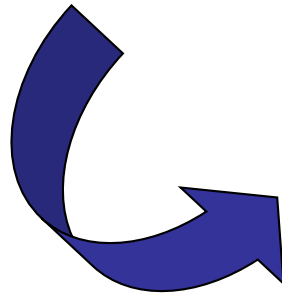
A star schema is :

- ✓ A set of **dimension tables** $DT_1, DT_2, \dots, DT_n$ - each corresponding to a dimension.
- ✓ Each DT_i is characterized by a primary key d_i and by a set of attributes describing the analysis dimensions with different aggregation levels
- ✓ A **fact table** FT that imports the primary keys of the dimensions tables. The primary key of FT is $d_1 d_2 \dots d_n$; FT contains also an attribute for each measure

Example of Star Schema transformed into a table structure



The colored attributes identify the primary event



What about aggregate data? We can include it in a unique fact table (other solutions later on)

1° row represents sale values for the single shop, 2° row represents aggregate values for Roma, 3° row represents aggregate values for Lazio, etc...

SALE

Shop_key	Date_key	Prod_key	qty	profit	...
1	1	1	170	85	...
2	1	1	300	150	...
3	1	1	1700	850	...
...	...	...	...	...	...

SHOP

Shop_key	shop	city	region	...
1	COOP1	Bologna	E.R.	...
2	-	Roma	Lazio	...
3	-	-	Lazio	...
...	...	...	...	...

Star schema: considerations

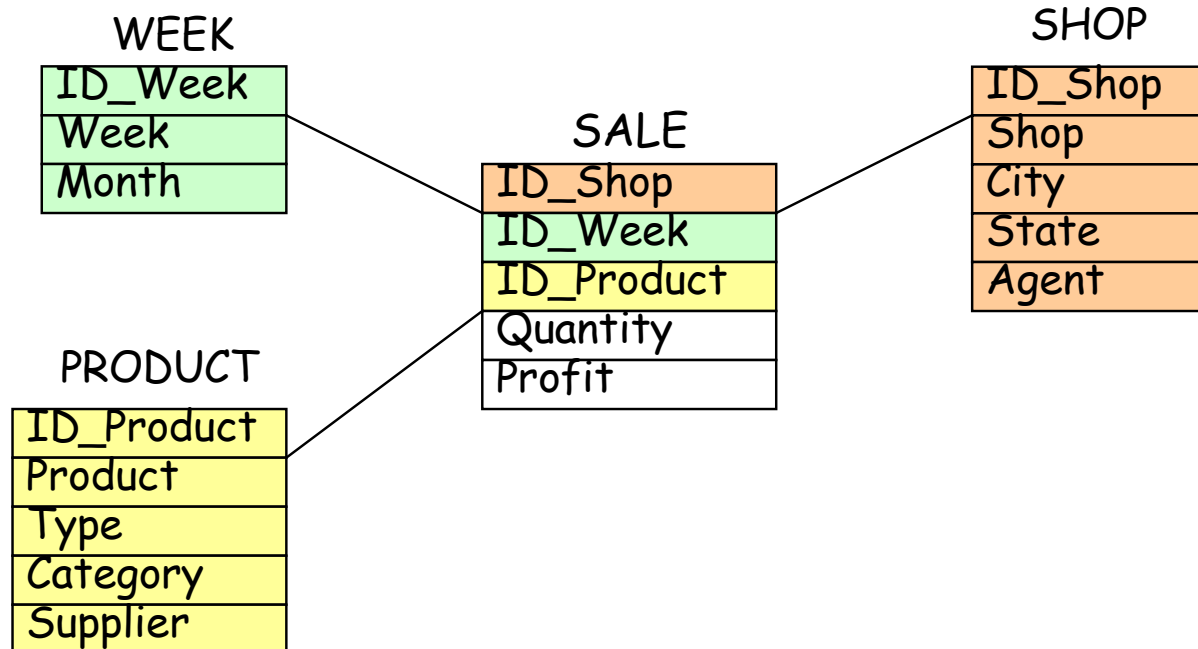
- The fact table contains information expressed at different aggregation levels
- Dimension table keys are generated ids (aka surrogates), for space efficiency reasons
- Dimension tables are de-normalized, e.g., note that

product → type → category

is a transitive dependency

- De-normalization introduces redundancy, but fewer joins to do (no prob because there are no data updates)

OLAP queries on Star Schema

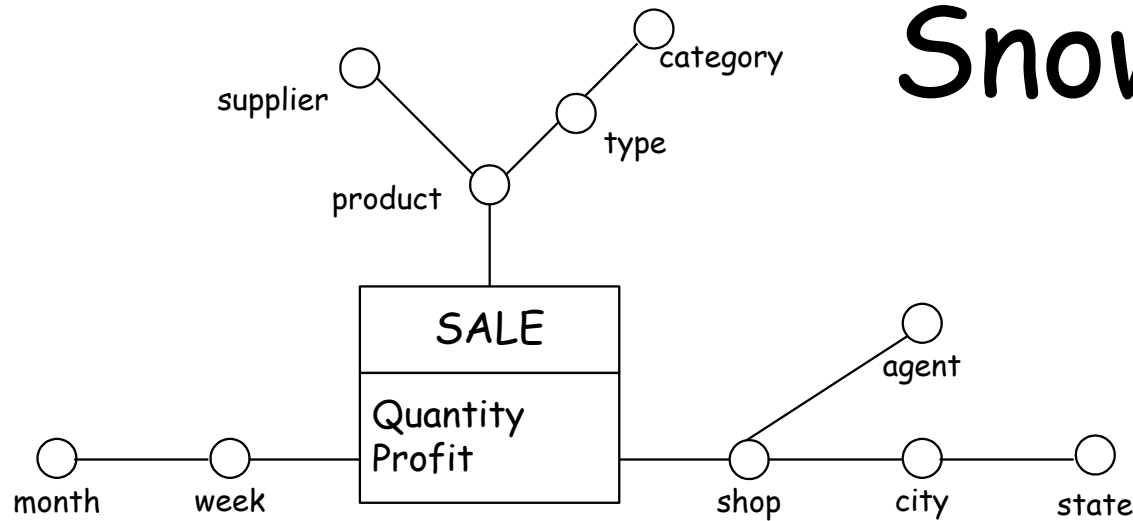


```
select City, Week, Type, sum(Quantity)
from Week, Shop, Product, Sale
where Week.ID_Week=Sale.ID_Week and
        Shop.ID_Shop=Sale.ID_Shop and
        Product.ID_Product=Sale.ID_Product and
        Product.Category = 'FoodStuff'
group by City, Week, Type
```

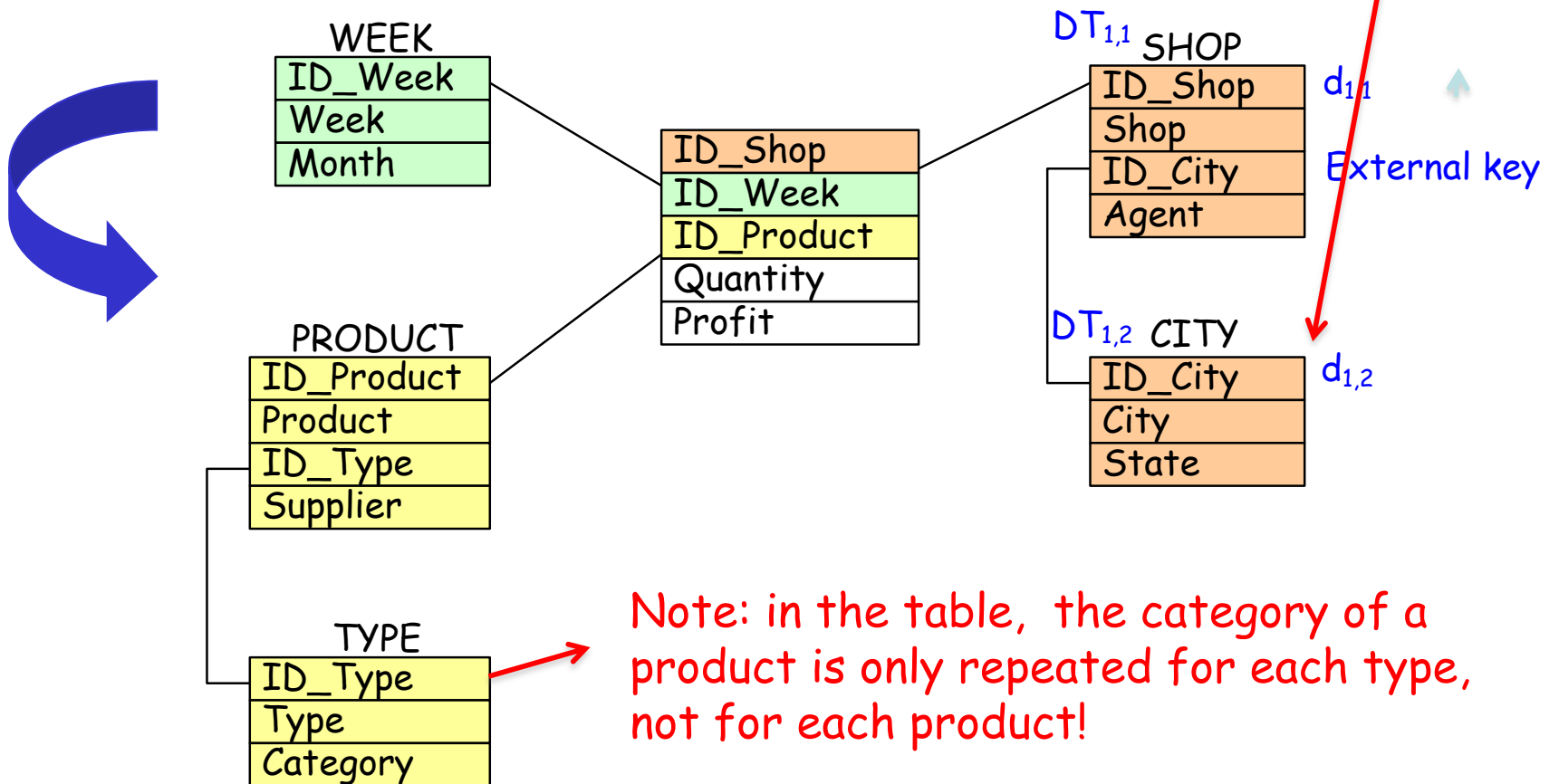
Snowflake schema

- The snowflake schema reduces the de-normalization of the dimensional tables DT_i of a star schema:
removal of some
transitive dependencies

Snowflake schema



Note: in the table, the state of a shop is only repeated for each city, not for each shop!



Snowflake schema

In a snowflake schema:

- The dimension tables are composed by
 - ✓ A primary key $d_{i,j}$
 - ✓ A subset of DT_i attributes that directly depend on $d_{i,j}$
 - ✓ Zero or more external keys that allow to obtain the entire information
- **Primary dimension tables:** their keys are imported in the fact table
- **Secondary dimension tables**

Snowflake Schema: considerations

- Reduction of memory space
- New surrogate keys
- Advantages in the execution of the queries related to attributes contained into fact and primary dimension tables

Normalization & Snowflake schema

- If there exists a cascade of transitive dependencies, the attributes depending (transitively or not) on the snowflake attribute are placed in a new table

SHOP	
ID_Shop	
Shop	
City	
Region	
State	
Agent	

$ID_Shop \rightarrow Shop$
 $Shop \rightarrow City$
 $City \rightarrow Region$
 $Region \rightarrow State$
 $Shop \rightarrow Agent$

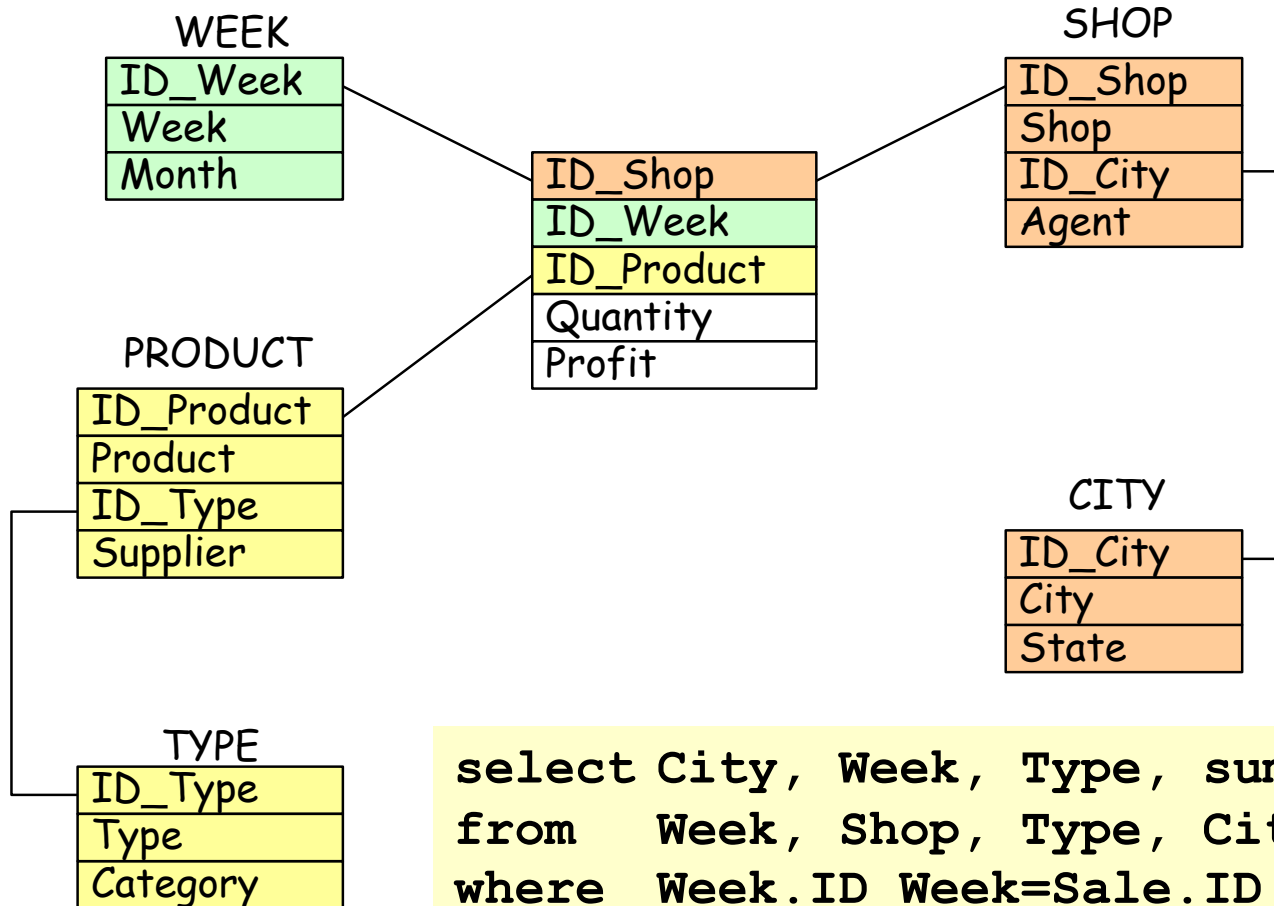
SHOP	
ID_Shop	
Shop	
ID_City	
Agent	

$ID_Shop \rightarrow Shop$
 $Shop \rightarrow ID_City$
 $Shop \rightarrow Agent$

CITY	
ID_City	
City	
Region	
State	

$ID_City \rightarrow City$
 $City \rightarrow Region$
 $Region \rightarrow State$

OLAP queries on snowflake schema



```
select City, Week, Type, sum(Quantity)
from Week, Shop, Type, City, Product, Sale
where Week.ID_Week=Sale.ID_Week and
Shop.ID_Shop=Sale.ID_Shop and
Shop.ID_City = City.ID_City and
Product.ID_Product=Sale.ID_Product and
Product.ID_Type=Type.ID_Type and
Product.Category = 'FoodStuff'
group by City,Week, Type
```

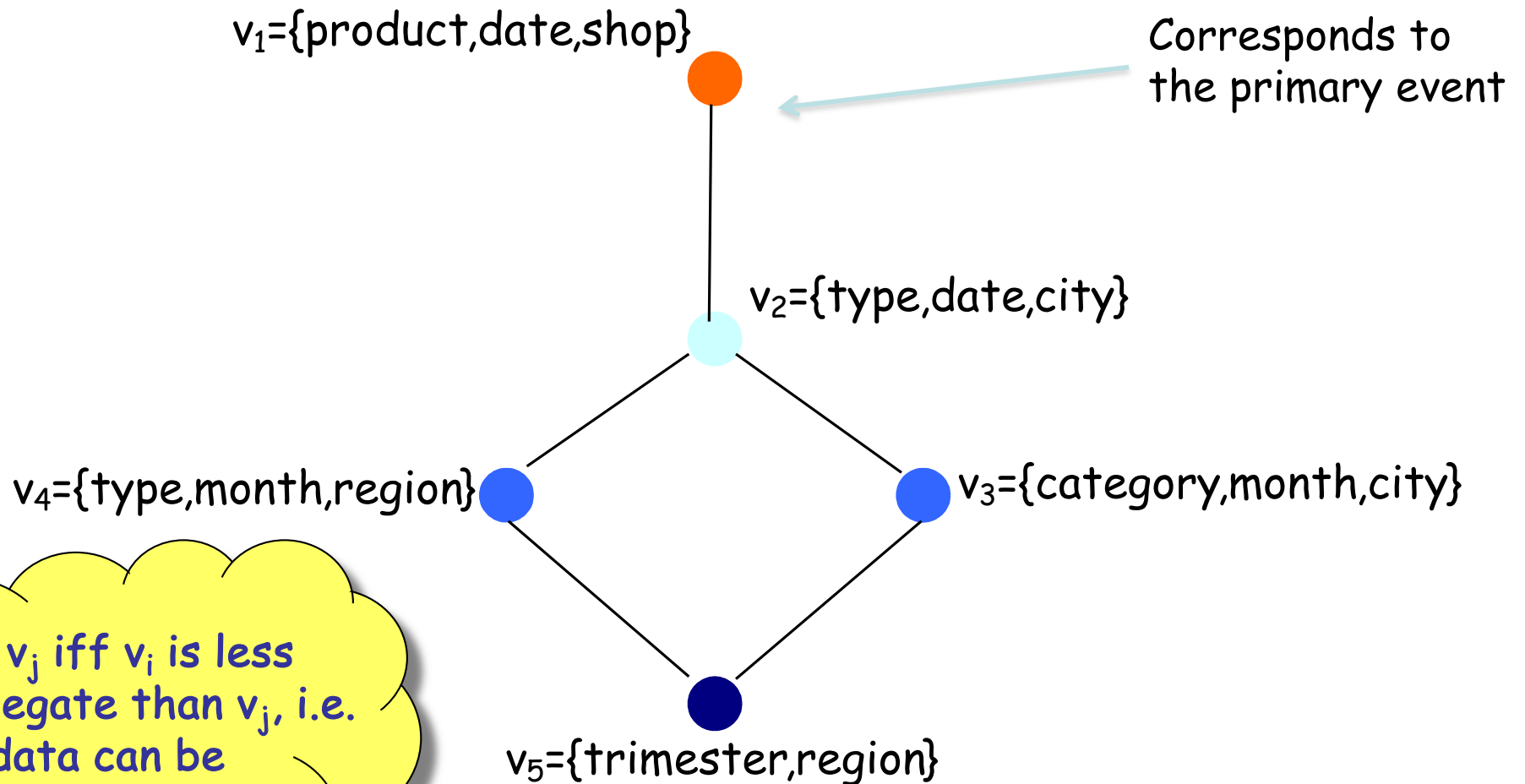
Materialized Views

- Aggregation allows to obtain summarized information
- Aggregation computation is very expensive → pre-computation
- Building materialized views corresponds to creating fact tables containing more aggregate data

Materialized Views

- A view can be characterized by its aggregation level (**pattern**)
 - **Primary views**: correspond to the primary aggregation levels
 - **Secondary views**: correspond to secondary aggregation levels ("secondary events")

Views (MultiDimensional Lattice)



$v_i \leq v_j$ iff v_i is less aggregate than v_j , i.e. v_j 's data can be computed from v_i 's data

Partial aggregations

- Sometimes it is useful to introduce new measures in order to manage aggregations correctly
 - **Derived measures**: obtained by applying mathematical operators to two or more values of the same tuple

Partial aggregations

$$\text{Profit} = \text{Quantity} * \text{Price}$$

Type	Product	Quantity	Price	Profit
T1	P1	5	1,00	5,00
T1	P2	7	1,50	10,50
T2	P3	9	0,80	7,20

22,70
(total profits)

SUM

AVG

Derived Table

Type	Quantity	Price	Profit
T1	12	1,25	15,00
T2	9	0,80	7,20

We can't just compute the profits as before!!

22,20

The correct solution consists in the aggregation of the data of the primary table

Aggregate operators

- **Distributive operator**: allows to aggregate data starting from partially aggregated data (e.g. sum, max, min)
- **Algebraic operator**: requires **further information** to aggregate data (e.g. avg)
- **Holistic operator**: it is not possible to obtain aggregate data starting from partially aggregate data (e.g. median)

Aggregate operators

- Currently, **aggregate navigators** are included in the commercial DW system
- They allow to re-formulate OLAP queries on the “best” view
- They manage aggregates **only by means of distributive** operators

Relational schema and aggregate data

- It is possible to define different variants of the star schema in order to manage aggregate data
- First solution (already seen): data of primary and secondary views are stored in the same fact table
 - NULL values for attributes having aggregation levels finer than the current one

Aggregate data in a unique fact table

1° row represents sale values for the single shop, 2° row represents aggregate values for Roma, 3° row represents aggregate values for Lazio, etc...

SALE

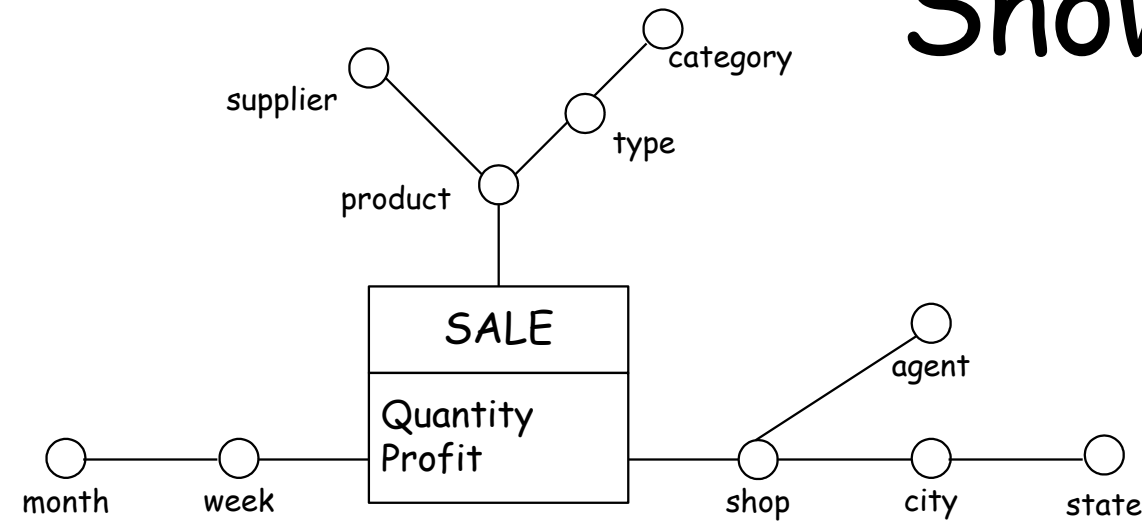
Shop_key	Date_key	Prod_key	qty	profit	...
1	1	1	170	85	...
2	1	1	300	150	...
3	1	1	1700	850	...
...	...	...	...	...	...

SHOP

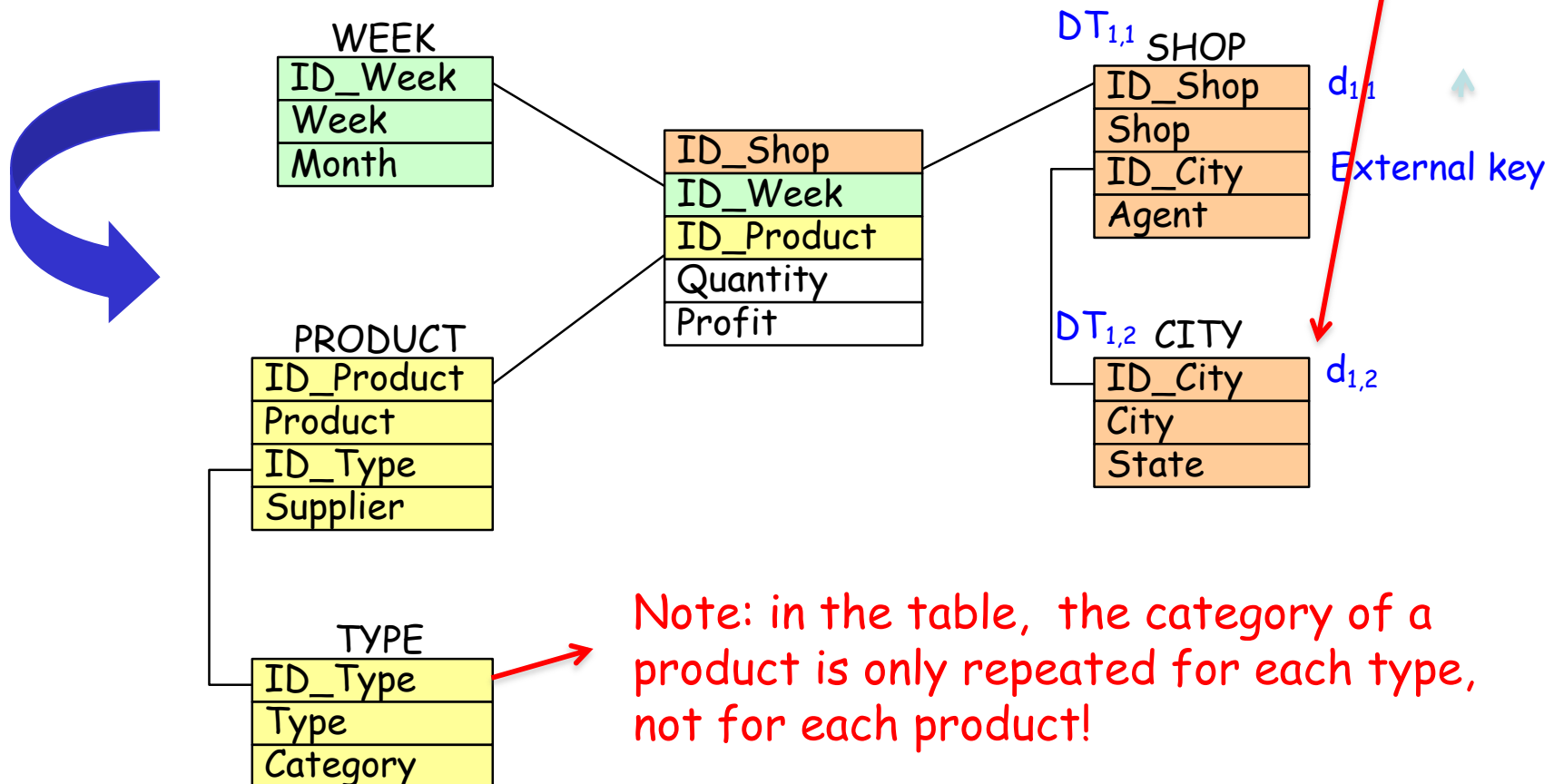
Shop_key	shop	city	region	...
1	COOP1	Bologna	E.R.	...
2	-	Roma	Lazio	...
3	-	-	Lazio	...
...	...	...	...	...

Null values!!

Snowflake schema (already seen)



Note: in the table, the state of a shop is only repeated for each city, not for each shop!



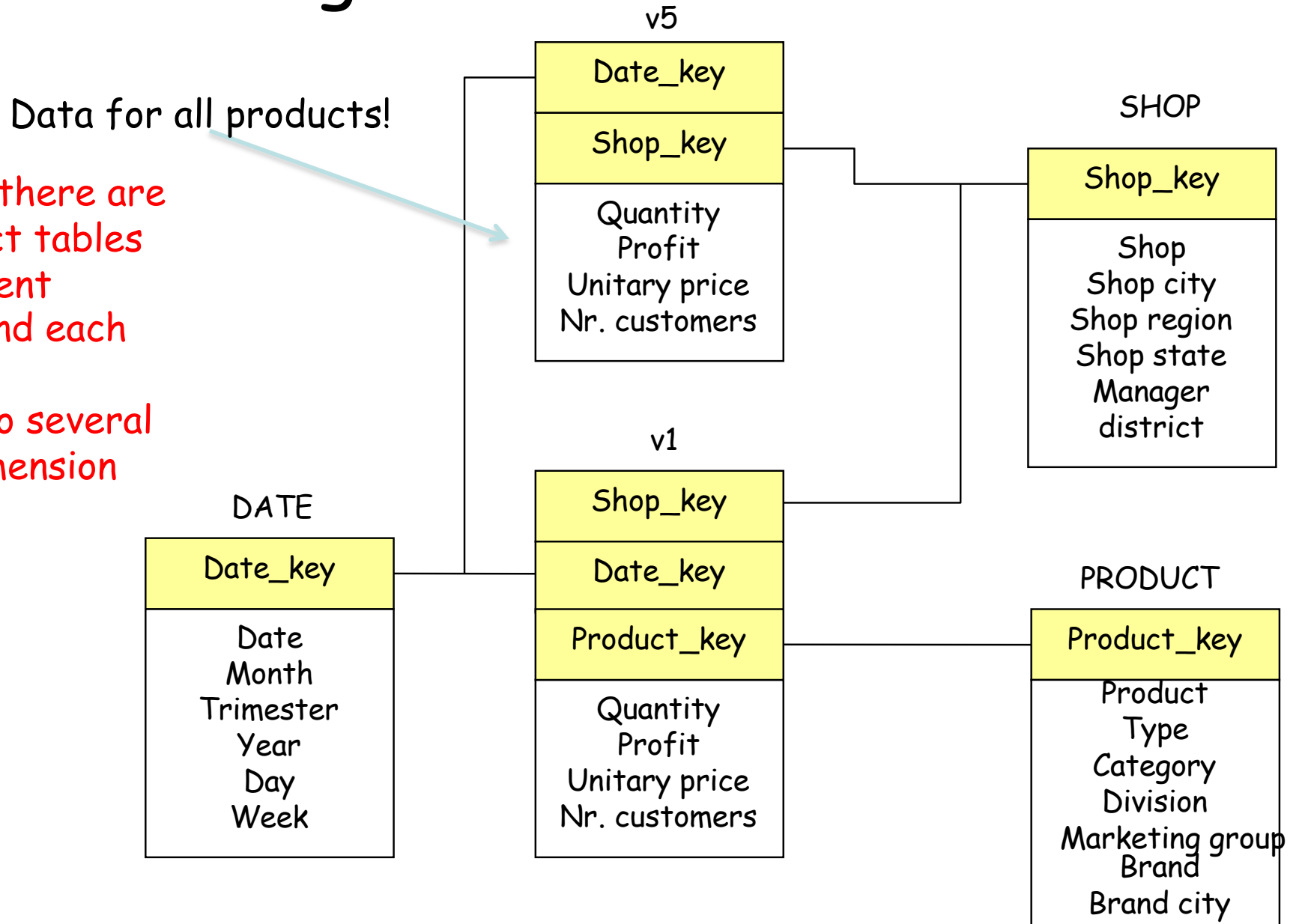
Note: in the table, the category of a product is only repeated for each type, not for each product!

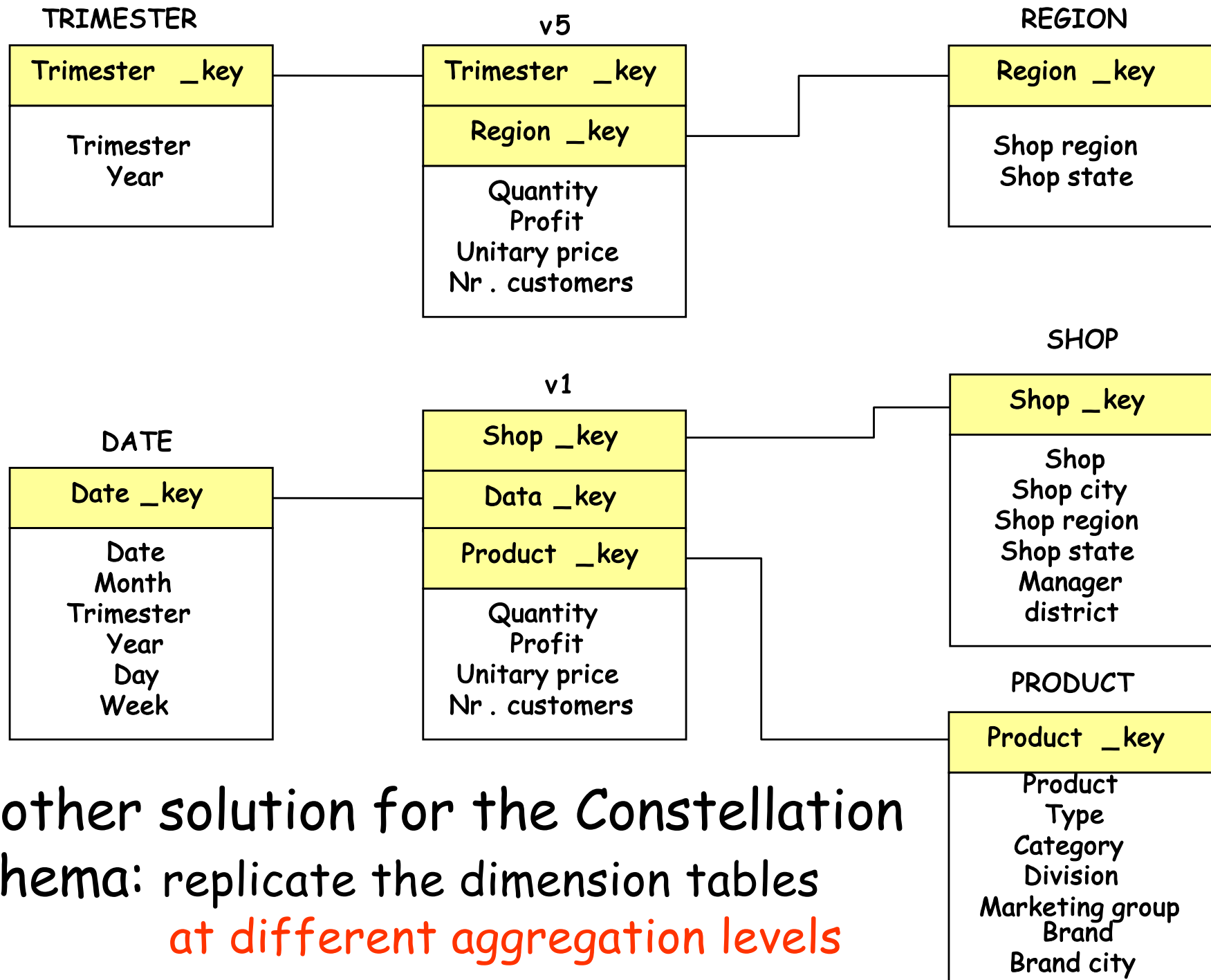
Relational schema and aggregate data

- **Another solution:** distinct aggregation patterns are stored in distinct fact tables: **constellation schema**
- Only the dimension of the fact table is reduced, but this is a great improvement
- Max optimization level: separate fact tables, and also **replicate dimension tables for different (the most used) aggregation levels**

Constellation Schema: multiple fact tables sharing dimension tables

Used when there are multiple fact tables with different measures and each fact table is related to several common dimension tables.





Another solution for the Constellation Schema: replicate the dimension tables
at different aggregation levels

Logical design
(ROLAP)

Logical modelling

- Sequence of steps that, starting from the conceptual schema, allow one to obtain the logical schema for a specific data mart

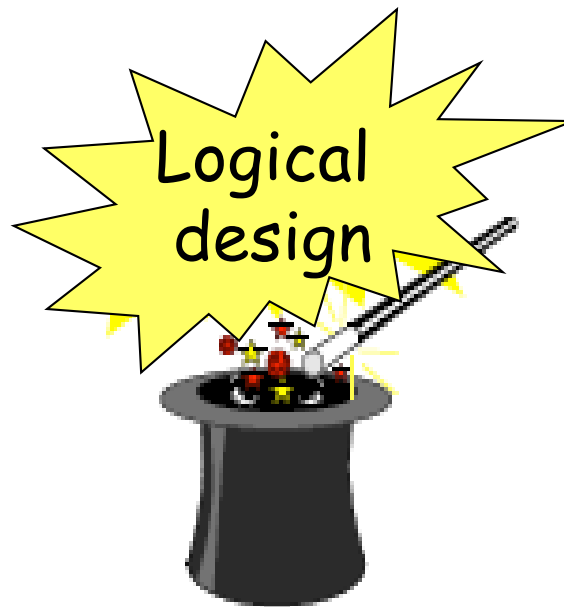
INPUT

Conceptual Schema

WorkLoad

Data Volume

System constraints



OUTPUT

Logical Schema

Workload

- With OLAP, the workload is **dynamic** in nature and intrinsically **extemporaneous**
 - Users' interests change during time
 - The number of queries grows when users gain confidence in the system
 - OLAP should be able to answer any (unexpected) request
- During the requirement collection phase, we can deduce the future workload from:
 - Interviews with users
 - Standard reports

Workload Prevision

- OLAP operations can be characterized:
 - Based on the required aggregation pattern
 - Based on the required measures
 - Based on the selection clauses
- At system run-time, the workload can be desumed from the system log

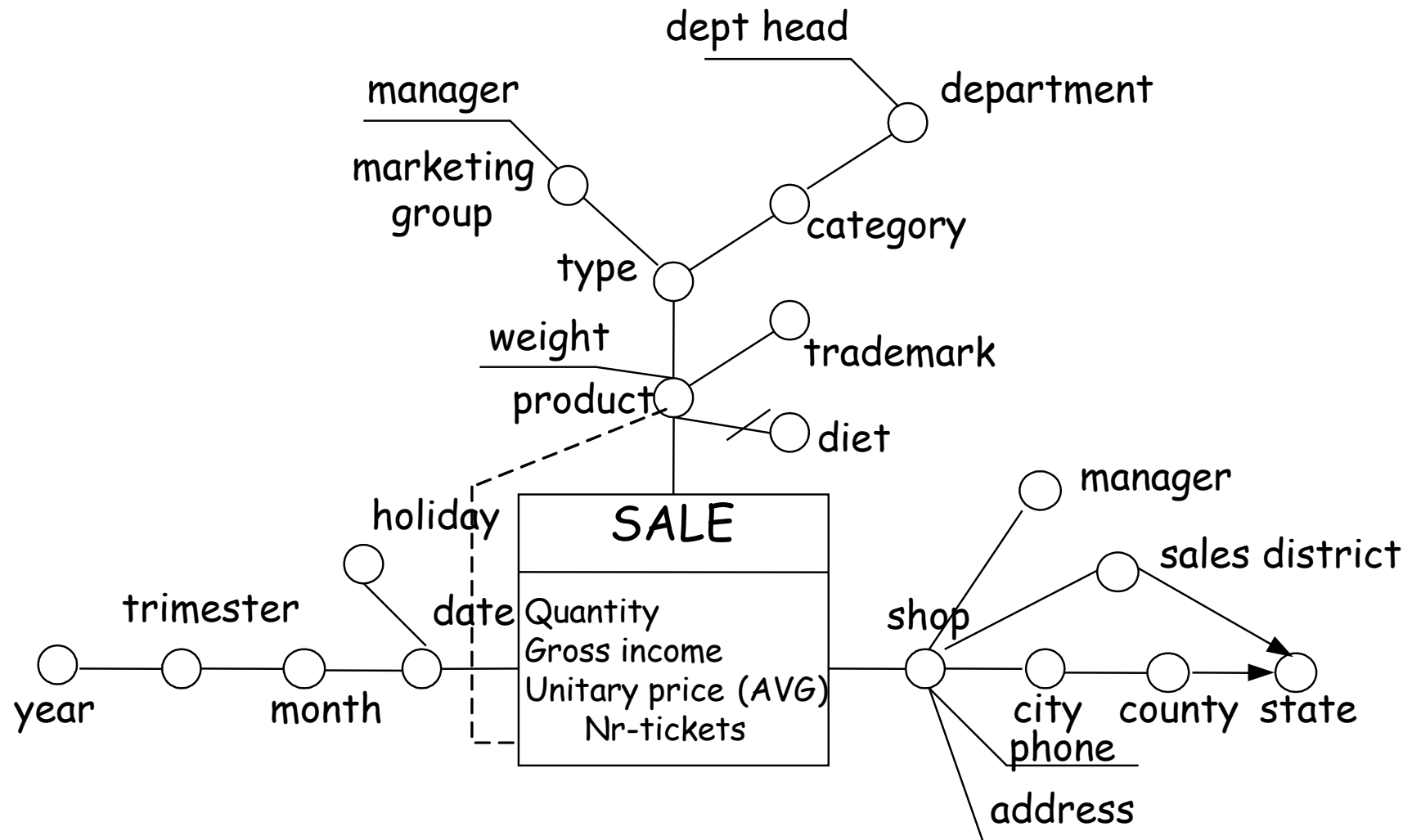
Data volume

- Depends on:
 - Number of distinct values for each attribute
 - Attribute size
 - Number of events (primary and secondary) for each fact
- Determines:
 - Table dimension
 - Index dimension
 - Access time

ROLAP Logical modelling: steps

1. Choice of the logical schema
(star/snowflake schema)
2. Conceptual schema translation
3. Choice of the materialized views
4. Optimization

Our fact schema example



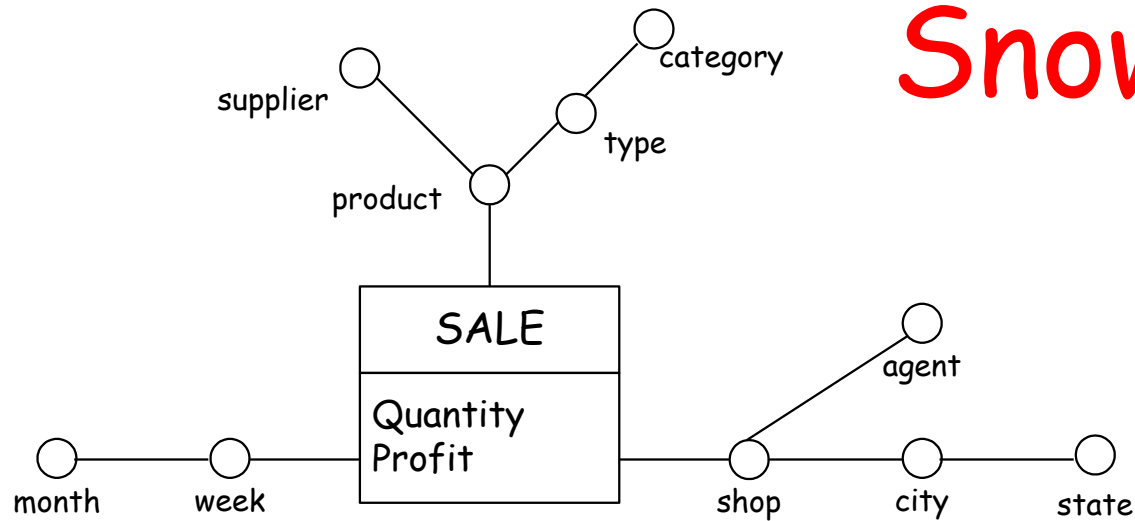
From fact schema to star schema

- Create a fact table containing measures and descriptive attributes directly connected to the fact
- For each hierarchy, create a dimension table containing all the attributes
- Seen in our example before

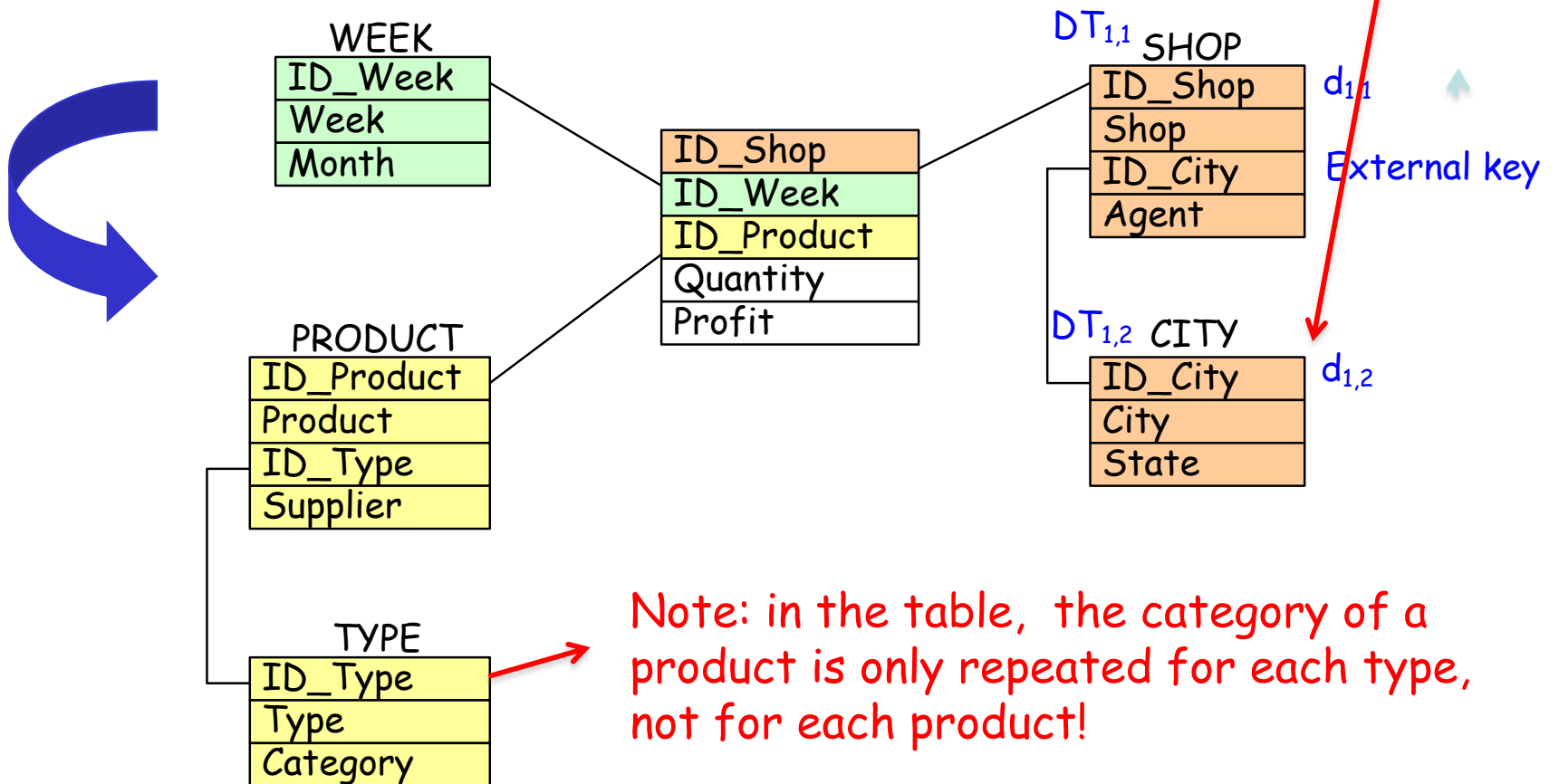
Guidelines

- Descriptive attributes (e.g. color)
 - If it is connected to a dimensional attribute, it has to be included in the dimension table containing the attribute (see next slide, snowflake example, agent)
 - If it is connected to a fact, it has to be directly included in the fact schema
- Optional attributes (e.g. diet)
 - Introduction of null values or ad-hoc values

Snowflake schema



Note: in the table, the state of a shop is only repeated for each city, not for each shop!



Other Guidelines

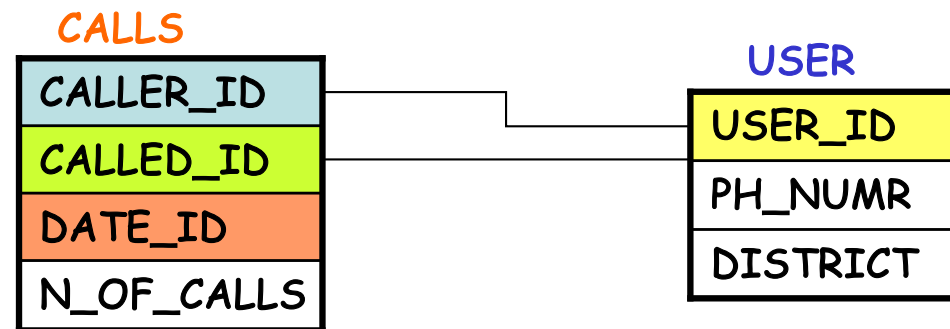
- Cross-dimensional attributes (e.g. VAT)
 - A cross-dimensional attribute b defines an N:M association between two or more dimensional attributes $a_1, a_2, \dots, a_k$
 - It requires to create a new table including b and having as key the attributes $a_1, a_2, \dots, a_k$

More Guidelines

- Shared hierarchies and convergence
 - A shared hierarchy is a hierarchy which refers to different elements of the fact table (e.g. caller number, called number)
 - The dimension table **should not** be duplicated
 - Two different situations:
 1. The two hierarchies contain the same attributes, but with **different meanings** (e.g. phone call → caller number, phone call → called number)
 2. The two hierarchies contain the same attributes **only for part of the hierarchy trees**

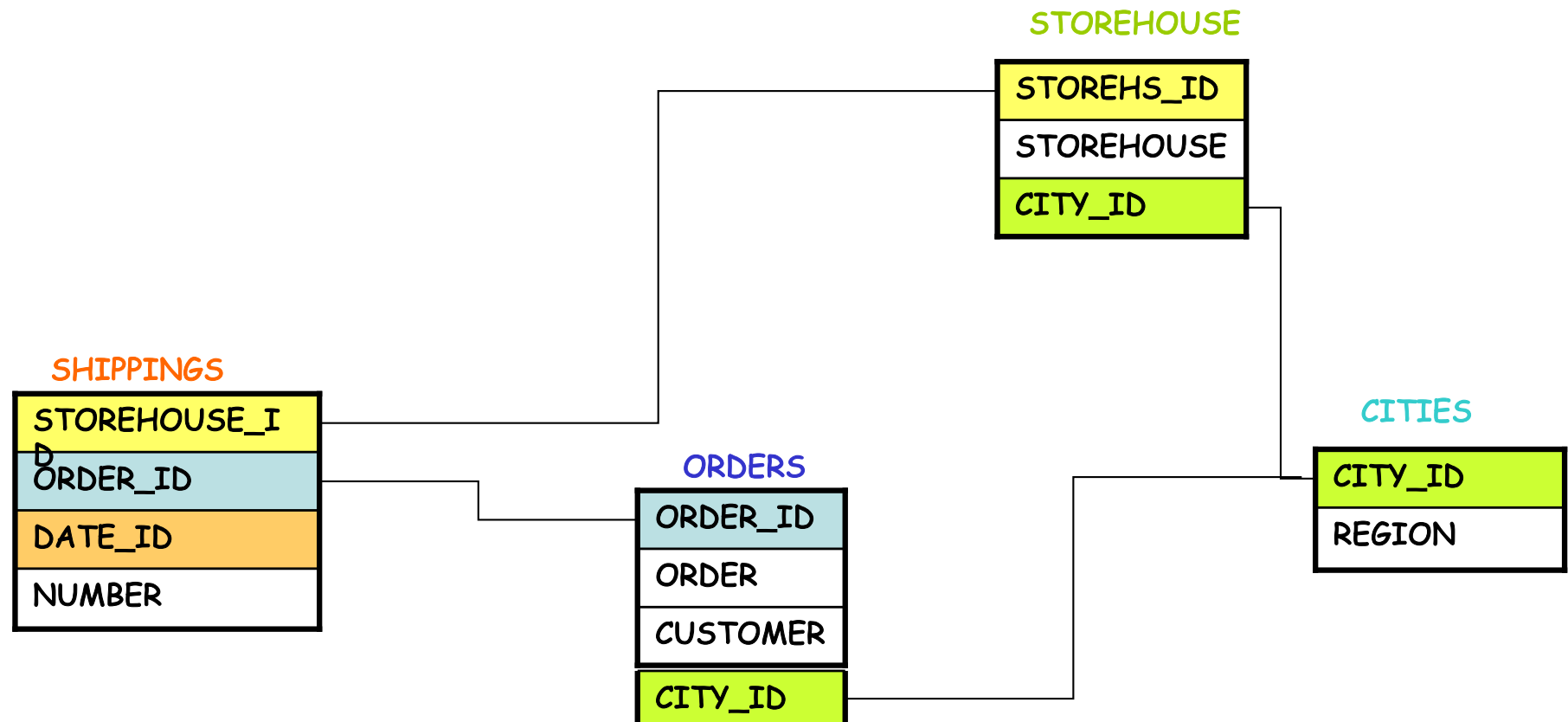
Shared hierarchies and convergence (1)

- The two hierarchies contain the same attributes, but **with different meanings** (e.g. phone call → caller number, phone call → called number)



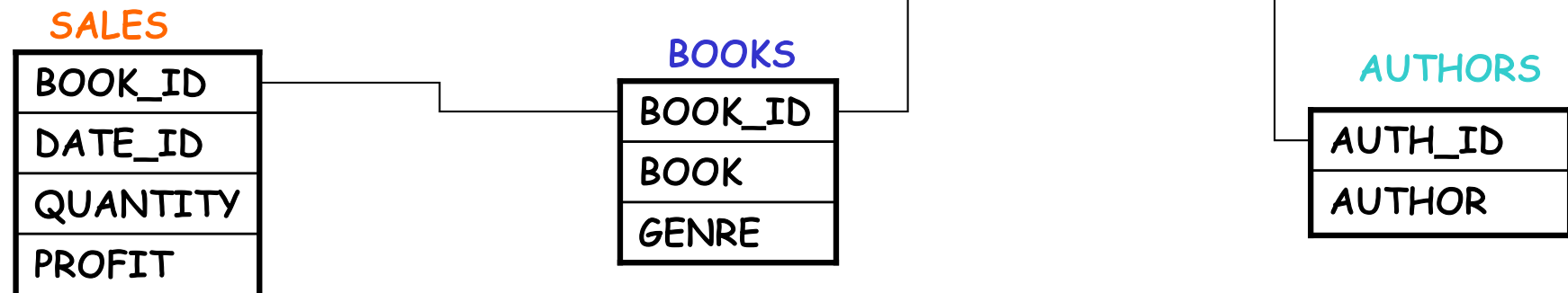
Shared hierarchies and convergence (2)

- The two hierarchies contain the same attributes **only for part of the hierarchy trees**. Here a city can refer to the storehouse or to the customer



When there is no functional dependency between two elements of the hierarchy

- We denoted it by multiple edges
 - A bridge table models the multiple edge
 - the key of the bridge table is composed by the combination of attributes connected to the multiple edge



Guidelines

- Multiple edges: bridge table
 - Weighed queries take into account the weight of the edge

Profit for each author

```
SELECT AUTHORS.Author, SUM(SALES.Profit * BRIDGE.Weight)
FROM AUTHORS, BRIDGE, BOOKS, SALES
WHERE AUTHORS.Author_id=BRIDGE.Author_id
AND BRIDGE.Book_id=BOOKS.Book_id
AND BOOKS.Book_id=SALES.Book_id
GROUP BY AUTHORS.Author
```

Guidelines

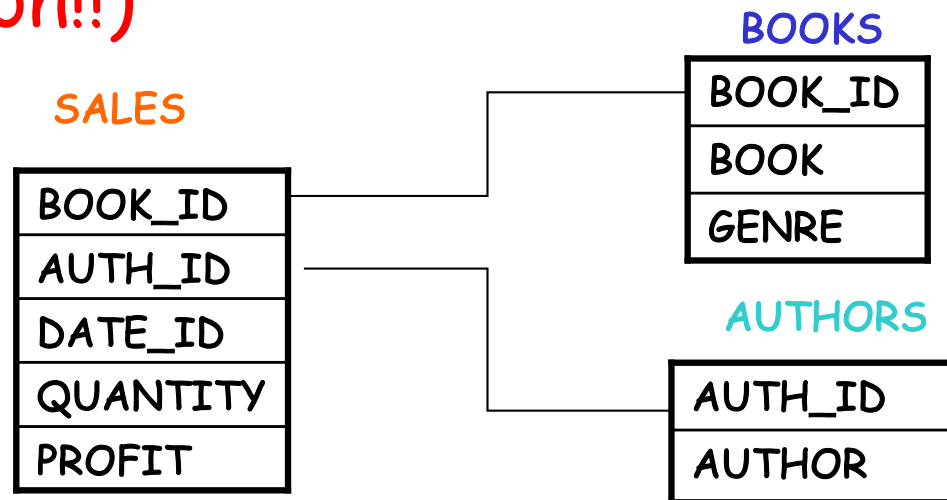
- Multiple edges: bridge table
 - Impact queries do not take into account the weight of the edge

Sold copies for each author

```
SELECT AUTHORS.Author, SUM(SALES.Quantity)
FROM AUTHORS, BRIDGE, BOOKS, SALES
WHERE AUTHORS.Author_id=BRIDGE.Author_id
AND BRIDGE.Book_id=BOOKS.Book_id
AND BOOKS.Book_id=SALES.Book_id
GROUP BY AUTHORS.Author
```

If we want to keep the star model

Multiple edges with a star schema: add authors to the fact schema (as another dimension!!)

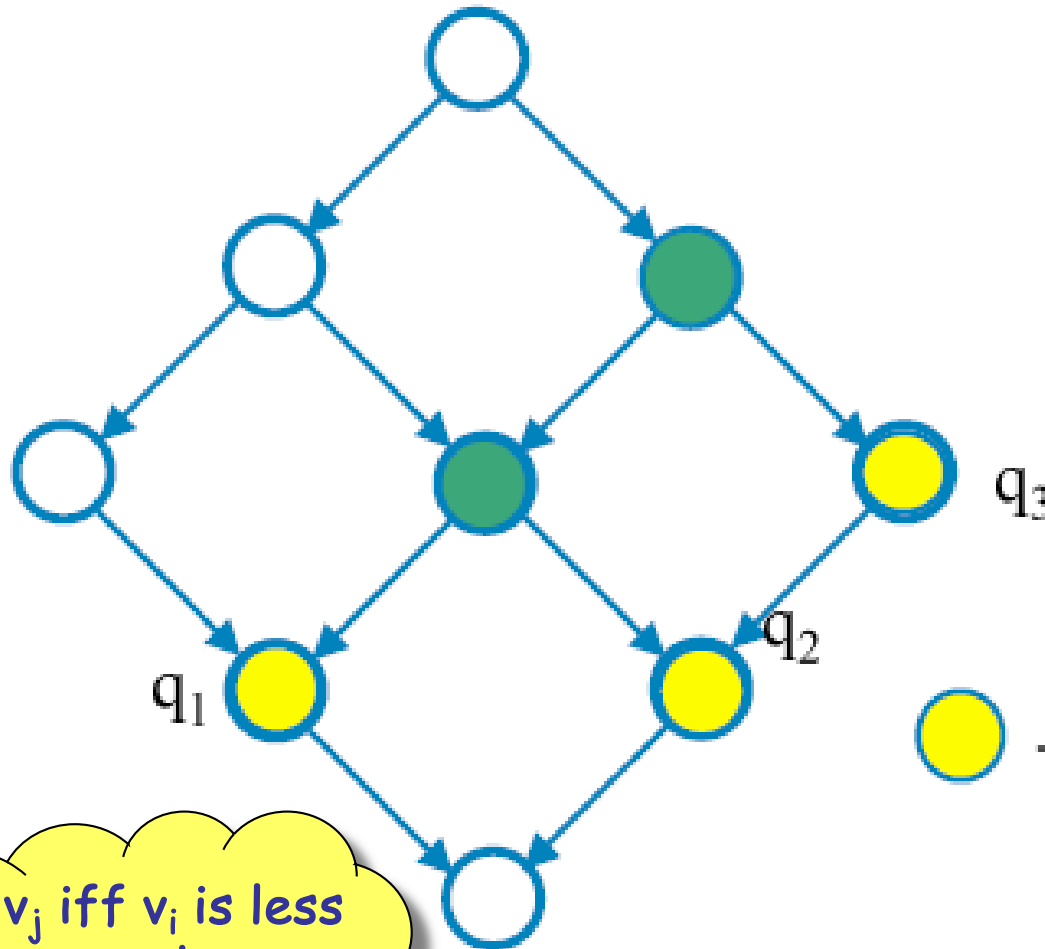


Here we don't need the weight because the fact table records quantity and profit per book and per author

Secondary-view precomputation

- The choice about the views that have to be materialized takes into account contrasting requirements:
 - **Cost functions' minimization**
 - Workload cost
 - View maintenance cost
 - **System constraints**
 - Disk space
 - Time for data update
 - **Users constraints**
 - Max answer time
 - Data freshness

Materialized views (MD lattice)



= exact views:

They solve exactly the queries



= less refined views:

They solve more than one query

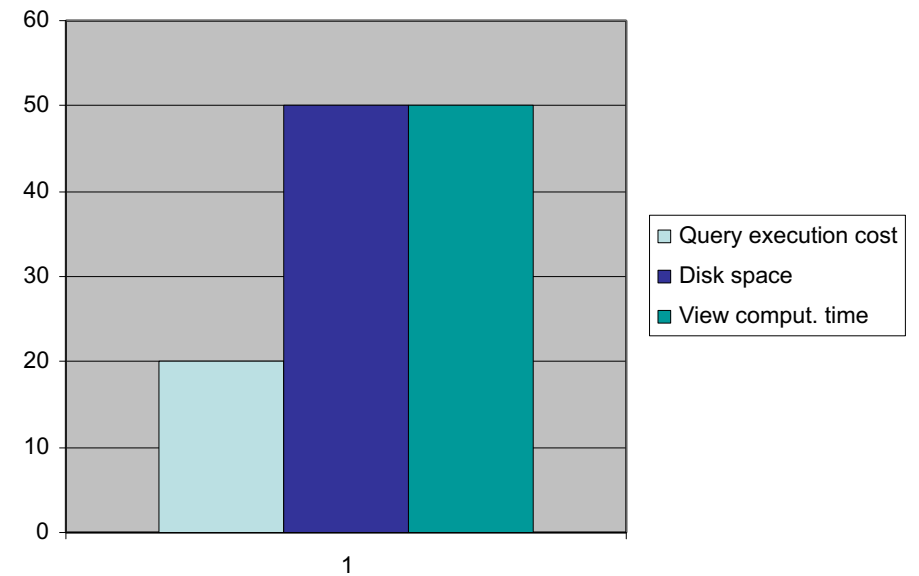
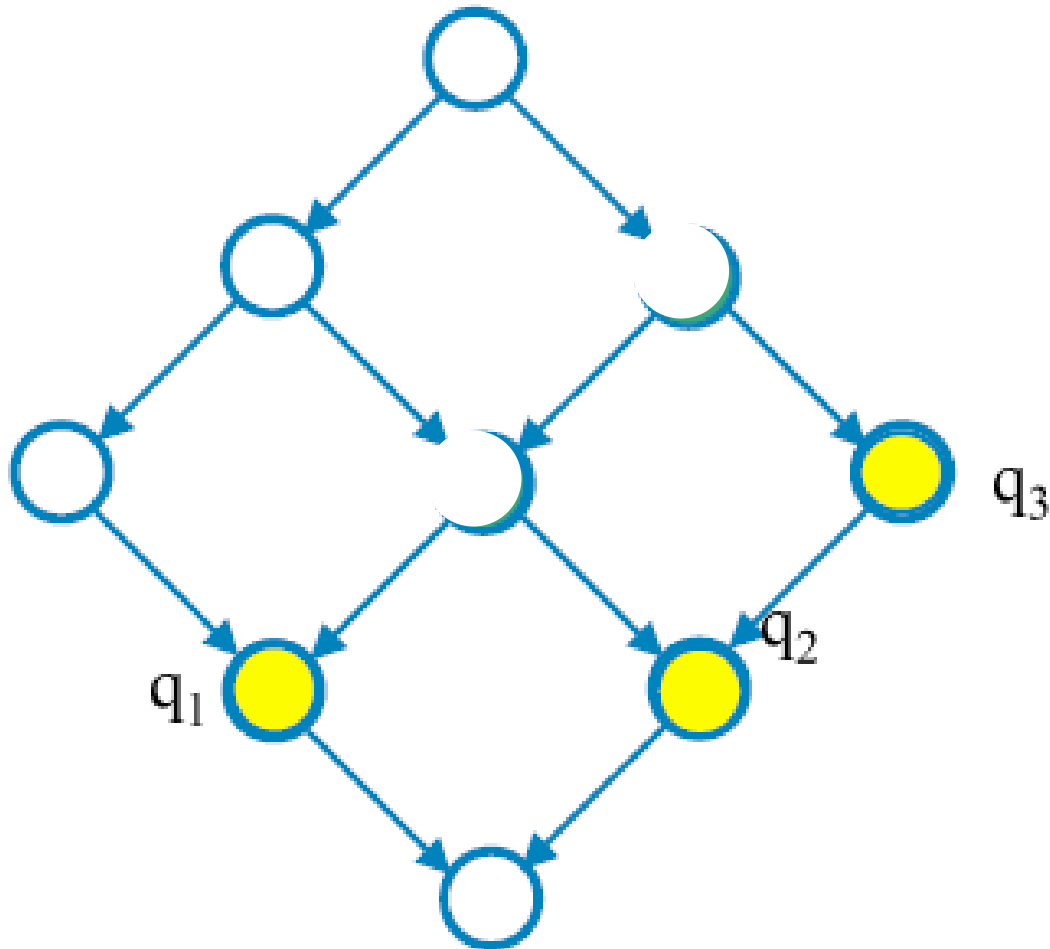


= candidate views

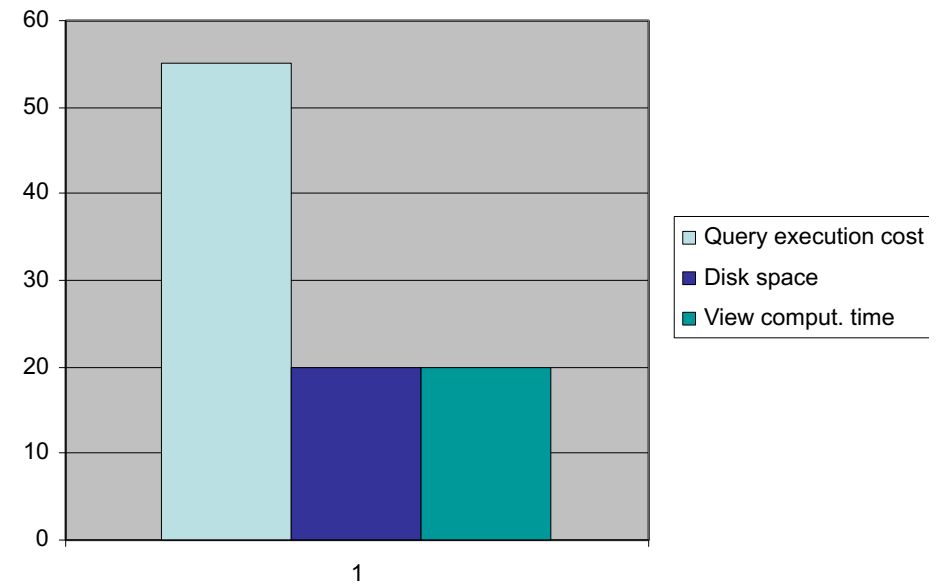
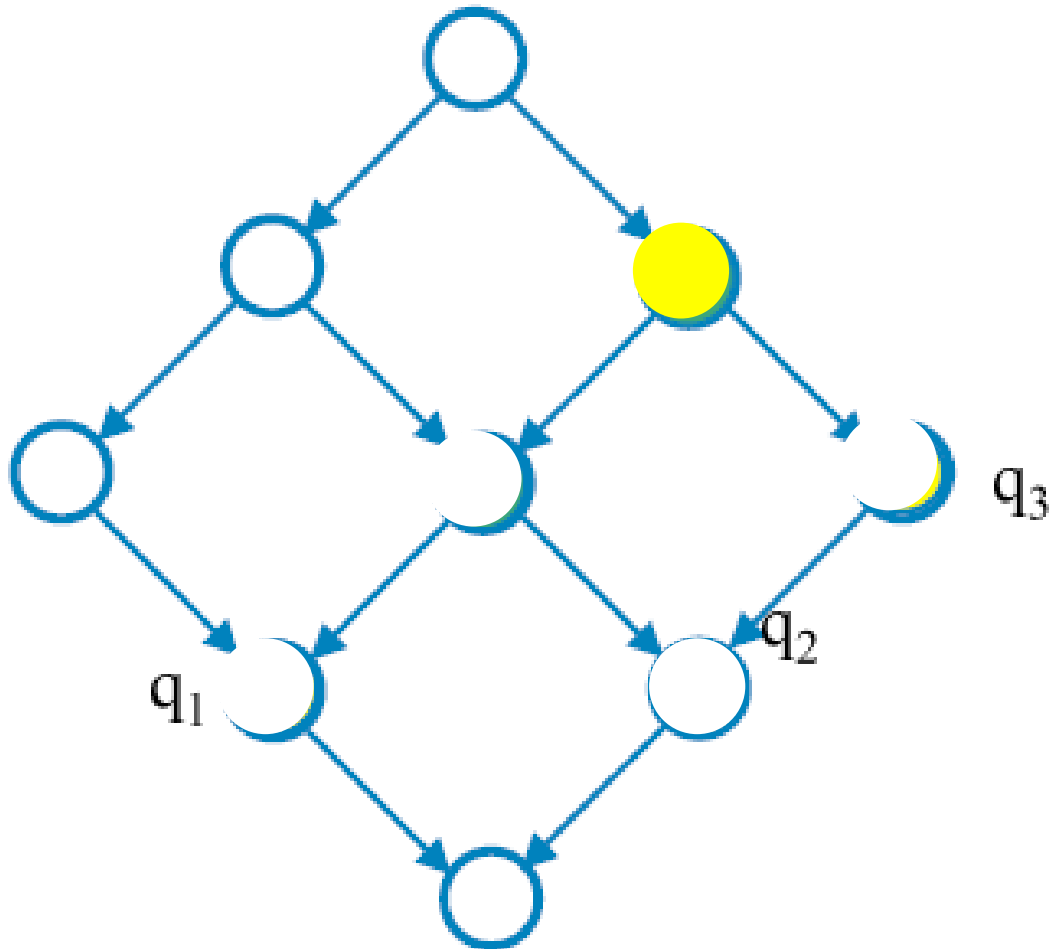
They could reduce
elaboration
costs

$v_i \leq v_j$ iff v_i is less aggregate than v_j ,
i.e. v_j 's data can be computed from v_i 's data

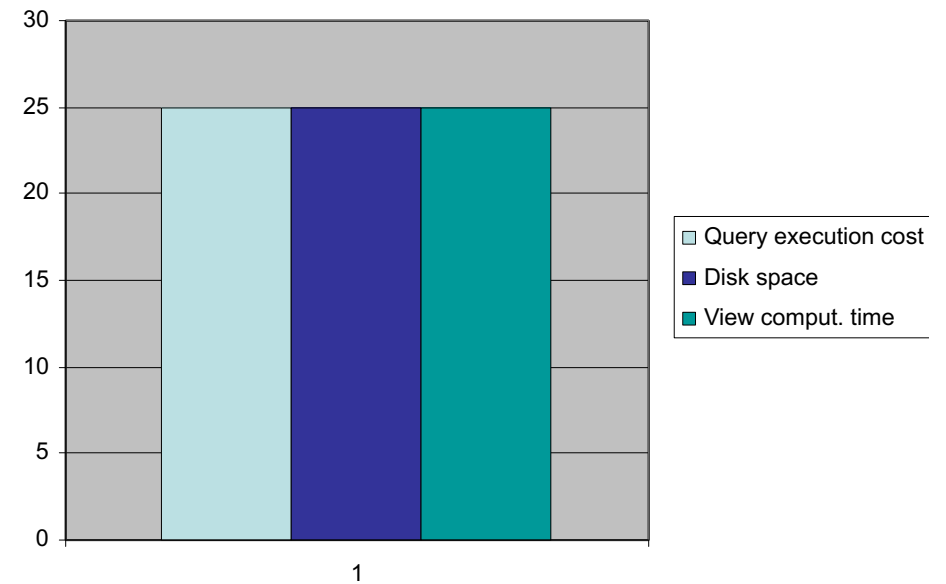
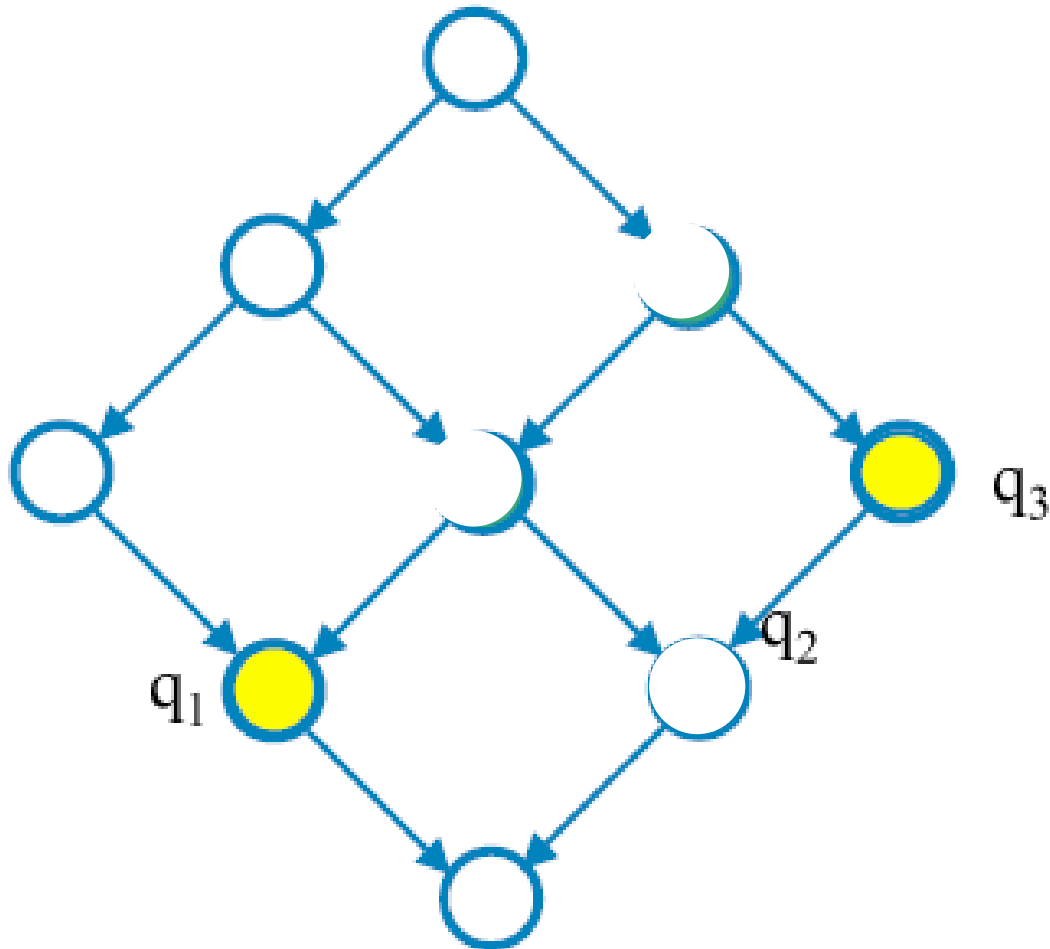
Materialized views (MD lattice)



Materialized views (MD lattice)



Materialized views (MD lattice)



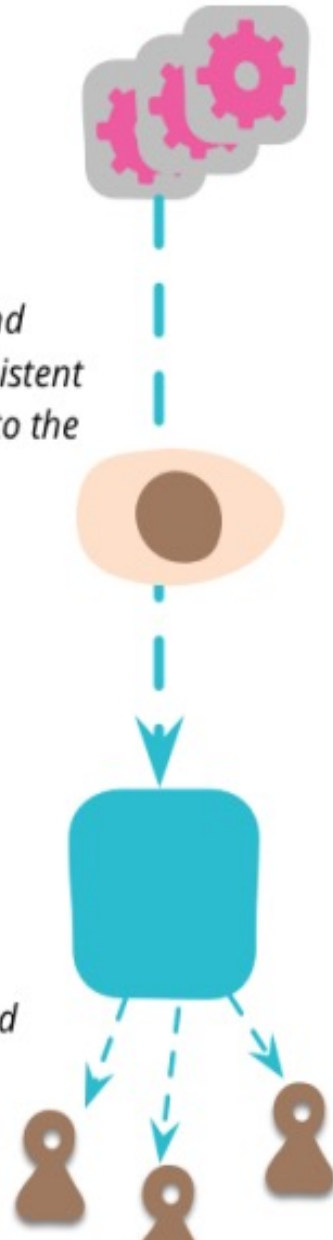
Materialized Views

- It is useful to materialize a view when:
 - It directly solves a frequent query
 - It reduce the costs of some queries
- It is not useful to materialize a view when:
 - Its aggregation pattern is the same as another materialized view
 - Its materialization does not reduce the cost

Data Warehouses vs. Datalakes

With a **data warehouse**, incoming data is cleaned and organized into a single consistent schema before being put into the warehouse...

... analysis is done directly on the curated warehouse data



With a **data lake**, incoming data goes into the lake in its raw form...

... we select and organize data for each need



DW vs Data Lakes

Data Warehouse	Dataspace
Usually Relational DBMS	Often HDFS/Hadoop
Data of recognized high value	Candidate data of potential value
Aggregated data	Detailed source data
Data conforms to enterprise standards	Fidelity to original format and condition (transformed/cleaned later, on demand, when needed)
Known entities	Raw material (metadata) to discover entities
Data integration up front	Data integration on demand
Schema on write	Schema on read (when doing analytics)

References

- [Stefano Rizzi](#) : Data Warehouse Design: Modern Principles and Methodologies McGraw-Hill, 2009
- M. Golfarelli, S. Rizzi: [Data Warehouse: teoria e pratica della progettazione](#) McGraw-Hill, 2002.
- Matteo Golfarelli: Data Warehouse Life-Cycle and Design. [Encyclopedia of Database Systems 2009](#): 658-664
- Stefano Rizzi: Business Intelligence. [Encyclopedia of Database Systems 2009](#): 287-288
- Ralph Kimball: [The Data Warehouse Toolkit: Practical Techniques for Building Dimensional Data Warehouses](#) John Wiley 1996.